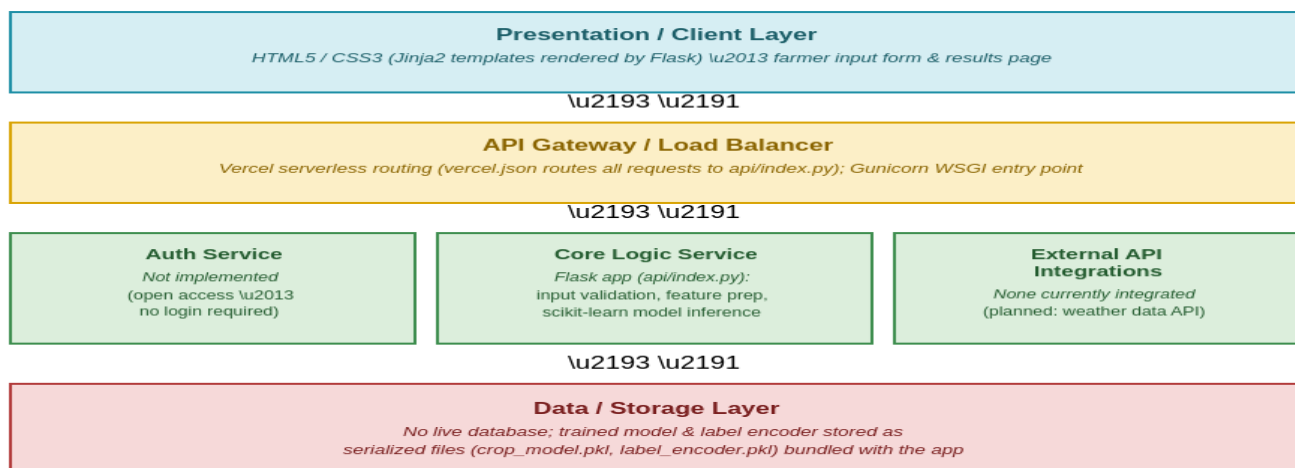


Solution Architecture

Date	1 July 2026
Team ID	SWTID-2026-5245
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Solution Architecture Diagram:

The diagram below reflects OptiCrop's actual deployed architecture: a single Flask application, hosted as a Vercel serverless function, that loads a pre-trained scikit-learn model to serve crop recommendations. There is currently no authentication, external API, or live database – these are noted as planned future additions.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the input form where farmers enter N, P, K, temperature, humidity, pH and rainfall, and displays the recommended crop / suitability result back to the user.	HTML5, CSS3, Jinja2 (Flask templates)
API Gateway	Routes all incoming HTTP requests to the single Flask application as a serverless function; no separate load balancer is used given the app's scale.	Vercel serverless routing (vercel.json), Gunicorn WSGI
Core Logic Service	Validates submitted form data, prepares it into the feature format expected by the model, runs inference using the trained classifier, and	Python, Flask, scikit-learn, NumPy, Pandas

	decodes the predicted label back into a crop name.	
Auth Service	Not implemented in the current version – the app is openly accessible without login, since it targets quick, low-friction use by farmers.	None (planned: simple role-based login for Farmer vs Researcher)
External API Integrations	No external APIs are currently called; all soil and climate values are entered manually by the user.	None currently (planned: weather data API for auto-fetching climate values)
Database / Storage	No live database is used. The trained model and label encoder are stored as serialized files bundled with the deployment, loaded into memory at request time.	Serialized .pkl files (crop_model.pkl, label_encoder.pkl)

SSSSSSSS

